Harness Engineering for Claude Code

A systematic guide to the infrastructure that makes AI sessions work

The model is 1.6% of what makes Claude Code effective. The harness is 98.4%. This repo teaches you to build it deliberately.

22/25

This repo's own harness audit score

Contents

Harness Engineering for Claude Code

> **One idea:** The infrastructure around an AI model — not the model itself — determines whether your sessions are productive or frustrating. This repo is a systematic guide to building that infrastructure.

The Big Idea

When Claude Code sessions go wrong — Claude edits the wrong file, repeats a mistake, asks for approval every five seconds, or drifts off-task — the instinct is to blame the model. The research says otherwise.

Analysis of Claude Code's own source code found that **1.6% is AI decision logic. The remaining 98.4% is the harness** — the permission pipeline, context management, tool router, and safety infrastructure wrapped around the model.

Four independent teams building coding agents from scratch converged on the same architecture. When teams independently invent the same structure, that's a signal it's a constraint imposed by the problem, not a design preference.

The Karpathy result made this concrete: a 65-line `CLAUDE.md` file with four behavioural rules moved AI coding accuracy from **65% → 94%**. The model didn't change. The harness did.

Harness engineering is the discipline of building that wrapper deliberately.

What a Harness Is

A Claude Code harness has four levers. You control all of them.

Lever	File	What it does
CLAUDE.md	`/CLAUDE.md`	Natural language context — what the project is, how to work in it, what not to touch
Permissions	`.claude/settings.json`	Which commands Claude can run without asking; what is always blocked
Hooks	`.claude/hooks/*.sh`	Shell scripts that fire at session start, before/after tool use, at session end
MCP Servers	`.claude/settings.json`	Subprocess tools that extend Claude's capabilities beyond built-ins

What Is In This Repo

```
Harness-engineering-plan/
│
├── research/               ← Seven in-depth guides on each harness component
│   ├── 00-overview.md      ← Start here: the conceptual model
│   ├── 01-claude-md-guide.md   ← How to write CLAUDE.md files that actually work
│   ├── 02-settings-guide.md    ← settings.json schema, permission syntax, examples
│   ├── 03-hooks-guide.md       ← Hook lifecycle, blocking hooks, shell patterns
│   ├── 04-mcp-guide.md         ← When and how to add MCP servers
│   ├── 05-project-types.md     ← Decision matrix: web app, API, CLI, data, monorepo
│   ├── 06-codebase-analysis.md ← How to audit and improve an existing project
│   └── notes/              ← Auto-saved research outputs from the agent
│
├── templates/              ← Ready to copy, fill in the blanks, and commit
│   ├── CLAUDE.md.template  ← All seven sections with placeholder guidance
│   ├── settings.json.template  ← Annotated settings with per-project-type sections
│   └── hooks/
│       ├── session-start.sh        ← Checks deps, env vars, services at session open
│       ├── pre-tool-use-safety.sh  ← Blocks force push, rm -rf, curl|bash
│       ├── post-edit-format.sh     ← Auto-formats .py/.ts/.go/.rs after every edit
│       └── stop-hook-git-check.sh  ← Reminds Claude to commit before stopping
│
├── examples/               ← Real-world CLAUDE.md files you can read and copy from
│   ├── karpathy-minimal/   ← The 65-line baseline: Think, Simplicity, Surgical, Goal
│   ├── web-app/            ← Next.js 14 + Tailwind + shadcn/ui
│   ├── api-service/        ← Python FastAPI + SQLAlchemy + Celery
│   └── data-pipeline/      ← dbt + Airflow + Snowflake
│
├── agent/                  ← Python research tools for exploring this repo
│   ├── main.py             ← Interactive REPL with stretch-loop research
│   ├── parallel.py         ← Run multiple research topics simultaneously
│   ├── audit.py            ← Score any project's harness from 0–25
│   ├── core.py             ← Two-pass Explore → Stretch research engine
│   ├── prompts.py          ← Research and literature-review prompts
│   └── tools.py            ← File, search, and note-saving tools
│
└── .claude/                ← This repo's own harness (score: 22/25)
    ├── settings.json       ← Permissions + hook registrations
    ├── hooks/              ← Four deployed hook scripts
    └── commands/           ← Four project slash commands
        ├── ultraplan.md    ← /ultraplan: deep parallel planning before any code
```

```
        ├── goal.md          ← /goal: set and track a session goal
        ├── agents.md        ← /agents: parallel research on any topic list
        └── ultrareview.md   ← /ultrareview: five-pass code review
```

Where to Start

You are new to harness engineering:
Read `research/00-overview.md` (10 min). It explains the mental model and why each lever exists.

You want to set up a new project right now:
1. Copy `examples/karpathy-minimal/CLAUDE.md` to your project root — this is the baseline
2. Fill in your project-specific sections from `templates/CLAUDE.md.template`
3. Copy `templates/settings.json.template` to `.claude/settings.json` and uncomment what applies
4. Add one hook: `templates/hooks/session-start.sh` → `.claude/hooks/session-start.sh`

You have an existing project and want to know how healthy its harness is:

```
pip install -r requirements-agent.txt
python agent/audit.py /path/to/your/project
```

This scores the harness across five layers (0–5 each) and gives you a prioritised fix list.

You want to research a specific harness topic interactively:

```
export ANTHROPIC_API_KEY=sk-ant-...
python agent/main.py
# Then ask: "What hooks should a Python FastAPI project use?"
```

The Karpathy Baseline

Before writing anything project-specific, start with these four rules. They are in `examples/karpathy-minimal/CLAUDE.md` and are the minimum viable harness for any project.

Rule	The mistake it prevents
Think Before Coding — state assumptions; ask when confused	Silently guessing wrong → wrong implementation
Simplicity First — minimum code; no speculative abstractions	Over-engineering a two-line fix into a framework
Surgical Changes — only touch what the task requires	"Improving" adjacent code and breaking it
Goal-Driven Execution — define done before starting	Endless iteration with no clear exit condition

Merge these into every CLAUDE.md you write. Then add your project-specific sections on top.

The Research Guides

Guide	What you'll learn
`00-overview.md`	The harness model, the four levers, why harness quality dominates model quality
`01-claude-md-guide.md`	The seven essential sections, anti-patterns (stale content, documenting obvious things), the Karpathy baseline, a readiness checklist
`02-settings-guide.md`	Full settings.json schema, permission syntax (`Bash(git log *)` vs `Bash(*)`), per-project defaults, the deny-list patterns that matter
`03-hooks-guide.md`	The four lifecycle events, how blocking hooks work (`{"decision": "block"}`), idempotency, keeping hooks fast
`04-mcp-guide.md`	When MCP beats Bash tools, how to register servers, common servers (GitHub, PostgreSQL, SQLite), building a custom one
`05-project-types.md`	Decision matrix across five project types — which harness components matter most for each
`06-codebase-analysis.md`	The audit-fix-verify loop, scoring matrix, common anti-patterns in real codebases, the repeating improvement workflow

---

The Templates

Template	Use it when
`templates/CLAUDE.md.template`	Starting a CLAUDE.md from scratch — fill in the `{{PLACEHOLDERS}}`
`templates/settings.json.template`	Setting up `.claude/settings.json` — uncomment the sections that apply
`templates/hooks/session-start.sh`	Automating dep install + env var checks when sessions open
`templates/hooks/pre-tool-use-safety.sh`	Blocking destructive commands before Claude can run them
`templates/hooks/post-edit-format.sh`	Auto-formatting every file Claude edits (Python, TypeScript, Go, Rust, shell)
`templates/hooks/stop-hook-git-check.sh`	Reminding Claude to commit before ending a session

How to install a hook:

```
mkdir -p .claude/hooks
cp templates/hooks/session-start.sh .claude/hooks/session-start.sh
chmod +x .claude/hooks/session-start.sh
# Edit the REQUIRED_VARS array for your project's env vars
```

Then register it in `.claude/settings.json`:

```
{
  "hooks": {
    "SessionStart": [{ "command": ".claude/hooks/session-start.sh" }]
  }
}
```

The Examples

All four examples are complete, real-world CLAUDE.md files you can read and adapt:

`examples/karpathy-minimal/` — The baseline

65 lines. Four behavioural rules derived from Andrej Karpathy's coding principles. Reported to improve AI coding accuracy from 65% → 94%. Start here.

`examples/web-app/` — Next.js 14 frontend

Covers: App Router structure, pnpm-only (delete package-lock.json if you see it), auto-generated `lib/api/` (never edit by hand), shadcn/ui CLI for component additions, Turbopack fallback, Mapbox HTTPS requirement.

`examples/api-service/` — Python FastAPI backend

Covers: async SQLAlchemy session scoping, Alembic autogenerate gaps (array columns, hypertables), Celery task testing with `.apply()` not `.delay()`, TimescaleDB vs plain Postgres distinction.

`examples/data-pipeline/` — dbt + Airflow

Covers: never `dbt run --target prod` locally, `dbt deps` after pulling packages.yml changes, `profiles.yml` not committed, Airflow DAG bag parse errors, Snowflake zero-copy cloning.

The Audit Tool

`agent/audit.py` scores any project's harness across five layers:

```
python agent/audit.py /path/to/your/project
python agent/audit.py . --save     ← saves report to research/notes/
```

```
  Layer                  Score   Bar
 ──────────────────────────────────
  CLAUDE.md               4/5    ████████▒
  settings.json           3/5    ██████▒▒▒
  Hooks                   1/5    ██▒▒▒▒▒▒▒
  Architecture Context    2/5    ████▒▒▒▒▒
  Environment & Secrets   2/5    ████▒▒▒▒▒

  Total                  12/25
```

It then outputs a prioritised list of exactly what to fix and how.
Run it repeatedly as you improve things — the score tracks your progress.

Scoring guide:

Total score	What it means
0–5	No harness — Claude is operating blind
6–10	Minimal harness — some context, no safety
11–15	Developing harness — foundational pieces in place
16–20	Good harness — productive sessions, safety covered
21–25	Strong harness — Claude operates with high autonomy and few mistakes

The Research Agent

```
pip install -r requirements-agent.txt
export ANTHROPIC_API_KEY=sk-ant-...
python agent/main.py
```

The agent uses Claude Opus 4.7 with a two-pass **stretch loop**:

1. **EXPLORE** — Claude reads repo files using tools, writes a draft answer
2. **STRETCH** — Claude critiques its own draft, finds what it missed, writes a final improved answer

The final answer is always better than the first draft.

```
> What hooks should a Python FastAPI project use?
> parallel: CLAUDE.md sections, hook patterns, MCP servers   ← runs 3 topics at once
> parallel                                                    ← runs 5 default topics
> any question  save                                          ← auto-saves the answer
```

Parallel research (`python agent/parallel.py`) fires multiple topics simultaneously using async API calls and shows a live status table. Results save to `research/notes/` as dated markdown files.

The Slash Commands

Once you have opened this repo in Claude Code, four custom commands are available from the `/` menu:

`/ultraplan <task>`

Before writing any code, this runs a four-phase research loop:
- Phase 1: three Explore agents run in parallel, each with a distinct focus (relevant files / reusable utilities / blast radius)
- Phase 2: designs the implementation with dependency-safe ordering
- Phase 3: writes a structured plan document to `/tmp/ultraplan-current.md`
- Phase 4: presents the plan and waits for your approval before touching anything

Use this for any task where getting the plan wrong would be expensive.

`/goal <goal description>`

Sets the session goal. Claude writes it to `.claude/session-goal.md` with verifiable completion criteria, checks the current git state, and then checks every subsequent action against the goal. At session end, it records what was accomplished.

`/agents <topic1, topic2, topic3>`

Runs parallel research on a comma-separated list of topics. Uses `python agent/parallel.py` if available, otherwise spawns Explore sub-agents directly. Synthesises all findings into a structured report and offers to save it.

`/ultrareview <file or directory>`

Five sequential review passes: Correctness → Security → Performance → Style → Test Coverage. Every finding is reported with file path, line number, severity (CRITICAL/MAJOR/MINOR/SUGGESTION), and a specific fix. Saves the full report to `research/notes/`.

The Audit-Fix-Verify Loop

The right way to improve any project's harness is iteratively, not all at once:

```
AUDIT → find the gaps
  ↓
FIX → address the highest-priority gap (one layer at a time)
  ↓
VERIFY → re-run the audit; score should go up
  ↓
repeat until score ≥ 18/25 or sessions feel right
```

Priority order — tackle gaps in this sequence:

1. CLAUDE.md exists at all
2. Karpathy baseline rules merged in
3. Destructive-command deny rules (force push, rm -rf, reset --hard)
4. SessionStart hook (dep + env checks)
5. Project-specific gotchas (stop repeating the same Claude mistakes)
6. PreToolUse safety guard
7. PostToolUse auto-formatter
8. Scoped allow list (tighten after a week of real sessions)
9. MCP servers (only when a clear friction point exists)
10. Stop hook

Key Findings from the Literature

These are the most important empirical results documented in `research/notes/harness-engineering-literature.md`:

The 98% rule: Claude Code's source is 1.6% AI decision logic. 98.4% is harness. If your sessions feel unreliable, the answer is almost certainly in your harness, not the model.

Independent convergence: Four teams building coding agents independently arrived at the same architecture: outer loop → tool execution → result capture, ~12 primitive tools, multi-stage permission pipeline. This is a constraint, not a preference.

Performance impact: On Terminal Bench 2.0, one team moved a coding agent from Top 30 to Top 5 by changing only the harness. The model was identical.

The Karpathy result: 65 lines, 4 rules, 65% → 94% accuracy. The model didn't change.

Contributing

This is a living research document. When you discover a useful pattern — a new hook technique, a gotcha in a project type not yet covered, a better permission pattern — add it:

1. Add the pattern to the relevant `research/0N-*.md` guide
2. If it's a reusable template, add it to `templates/`
3. If it's a new project-type example, add a `examples/your-type/CLAUDE.md`
4. Update the status checklist in `CLAUDE.md`

Follow the existing heading and structure conventions in each file. The goal is that every entry is immediately usable by someone who reads it, not just conceptually interesting.

This Repo's Own Harness Score

Run `python agent/audit.py .` to see the current score and what remains to improve.